Conditional Statements

CSL 100

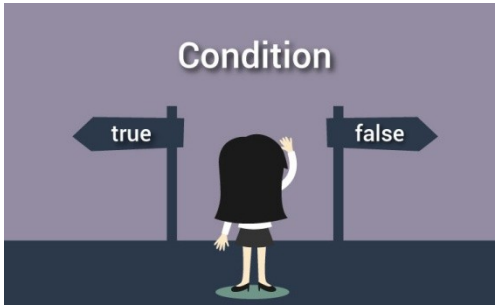
August 21, 2024

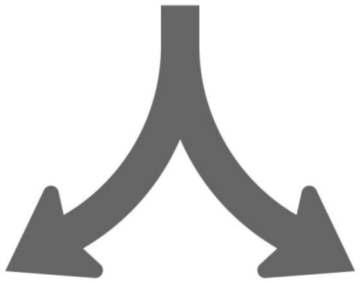
This Class

- Control statements
  - Branching – if-else

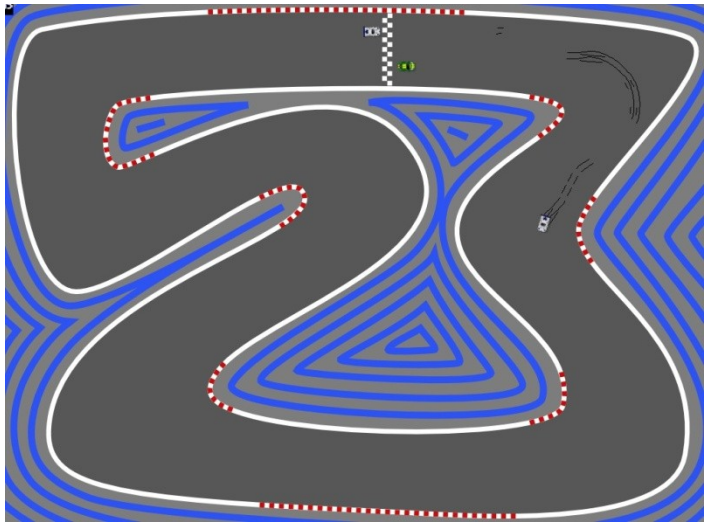
Control Statements

- Branching





- Looping



Control Statements

- Flow of control so far
  - Top to bottom

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  int n;
  double m;

  scanf(“%d”,&n);
  m = log(n);                  // natural log
  printf("%f\n", m);

  return 0;
}
```

Branching: Use Case

- What happens when n is negative?

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  int n;
  double m;

  scanf(“%d”,&n);
  m = log(n);                  // natural log
  printf("%f\n", m);

  return 0;
}
```

Goto Statement

- Can ask program to skip to a particular arbitrary location in your code
- Syntax –
  - *goto label;*
  - *label: expression;*
- Program control goes to the statement beginning with the label in the goto statement

Solving the Use Case

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  int n;
  double m;
  printf(“Please enter a positive number: ”);
  scanf(“%d”,&n);
  if (n<0)
       goto error;
  m = log(n);                  // natural log
  printf("%f\n", m);

  error:  printf(“Why can’t you follow instructions?");
  return 0;
}
```

Goto Statement

- A computer programmer is someone who, when asked to go to hell, objects to the use of the word ‘goto’
- (Almost) never use goto
  - Too arbitrary, makes code hard to understand



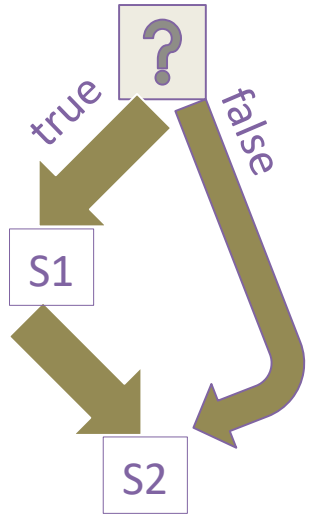
Branching Statements in C

- 3 types of conditional statements in C
  - if (cond) action
  - if (cond) action
    else some-other-action
  - switch-case
- Each action is a sequence of one or more statements!

If Statement

- General form of the if statement

```
if (expression)
        statement S1
statement S2
```

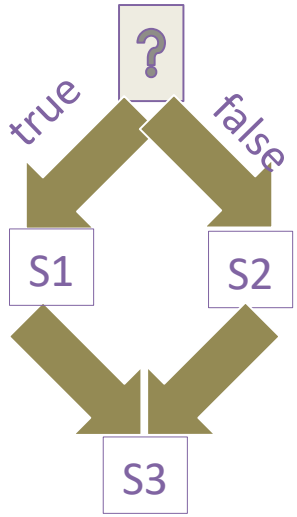


- Execution of if statement
  - First the expression is evaluated.
  - If it evaluates to a non-zero value, then S1 is executed and then control (program counter) moves to the statement S2.
  - If expression evaluates to 0, then S2 is executed.

if-else Statement

- General form of the if-else statement

```
if (expression)
        statement S1
else
        statement S2
statement S3
```



- Execution of if-else statement
  - First the expression is evaluated.
  - If it evaluates to a non-zero value, then S1 is executed and then control (program counter) moves to S3.
  - If expression evaluates to 0, then S2 is executed and then control moves to S3.
  - S1/S2 can be **block** of statements!

Solving the Use Case with if-else

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  int n;
  double m;
  printf(“Please enter a positive number: ”);
  scanf(“%d”,&n);
  if (n>0)
       m = log(n);                  // natural log
       printf("%f\n", m);
  else
       printf(“Why can’t you follow instructions?");

return 0;
}
```

Something’s not quite right in this code

Compound Statements

- A block of code containing zero or more statements
- Contained between { and }
- Format:

```
{
  Local Declarations

  Statements
}
```

Getting it Right

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  int n;
  double m;
  printf(“Please enter a positive number: ”);
  scanf(“%d”,&n);
  if (n>0){
       m = log(n);                  // natural log
       printf("%f\n", m);
  }
  else
       printf(“Why can’t you follow instructions?");
  return 0;
}
```

Good Indentation Practice

- This is a main statement
  - This is a dependent statement
- Main statements are statements in the main control flow of your program
  - Dependent statements branch off from the main flow
  - **Indent them**, for easier understanding of code
  - Matters more in some languages, like python
- Pro-tip: use 4 spaces instead of tab to indent

Example

- Problem: Input a, b, c are real positive numbers such that c is the largest of these numbers. Print ACUTE if the triangle formed by a, b, c is an acute angled triangle and print NOT ACUTE otherwise.

```
int main() {
      float a; float  b; float  c;
                scanf("%f%f%f", &a,&b,&c);        /* input a,b,c */

      if ( (a*a + b*b) >  (c*c) )  {  /* expression*/
          printf("ACUTE");
      }
      else {
          printf("NOT ACUTE");
      }
      return 0;
}
```

if-else Statement

- Read two integers and print the min.

```
# include <stdio.h>
int main() {
        int x, y;
        scanf("%d%d",
                        &x,&y);
    if (x < y) {
        printf("%d", x);
    } else {
        printf("%d", y);
    }
    return 0;
}
```

1. Check if x is less than y.
2. If so, print x
3. Otherwise, print y.

Tracing Execution of if-else

```
# include <stdio.h>
int main() {
        int x; int y;
        scanf("%d%d", &x,&y);
        if (x < y) {
                printf("%d\n",x);
        }
        else {
        printf("%d\n",y);
    }
    return 0;
}
```

Run the program

Input

6 10

x

6

y

10

Output

6

6 < 10 so the if-branch is taken

Nested if-else

```
#include <stdio.h>
#include <stdlib.h>

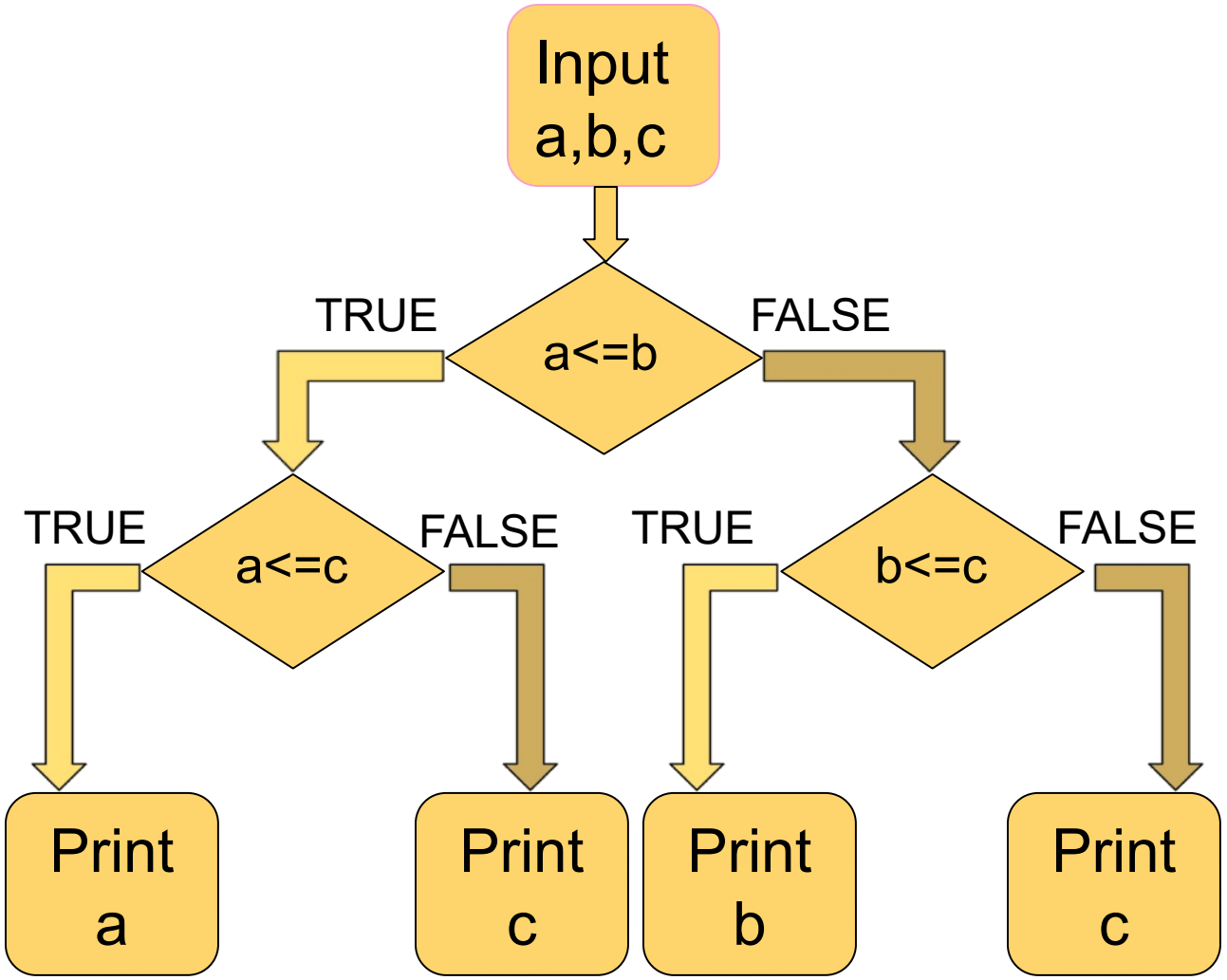
int main() {
    int c,d;
    printf("Do you like C? (0/1) ");
    scanf("%d",&c);
    if(c){
        printf("Do you really like C? (0/1) ");
        scanf("%d",&d);
        if(d)
            printf("You really like C!!");
        else
            printf("Not really");
    }
    else{
         printf("Ohh, You hate C!!");
    }

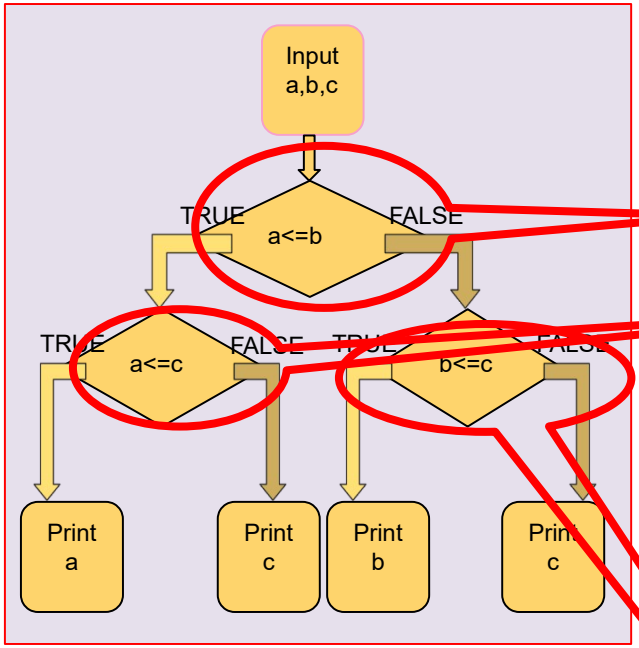

    return 0;
}
```

Nested if-else

- If e1 s1 else if e2 s2
- If e1 s1 else if e2 s2 else s3
- If e1 if e2 s1 else s2 else s3
- If e1 if e2 s1 else s2
- Rule of thumb: *else* clause looks for the closest previous *if* without an *else*

Finding min of 3 numbers






```
int a,b,c;
scanf("%d%d%d",&a,&b,&c);
if (a <= b) {
      if (a <= c) {
            printf("min = %d",a);
      }
      else {
            printf("min = %d", c);
      }
}
else {
      if (b <= c)  {
            printf("min = %d", b);
      }
      else {
            printf("min =%d", c);
      }
}
```

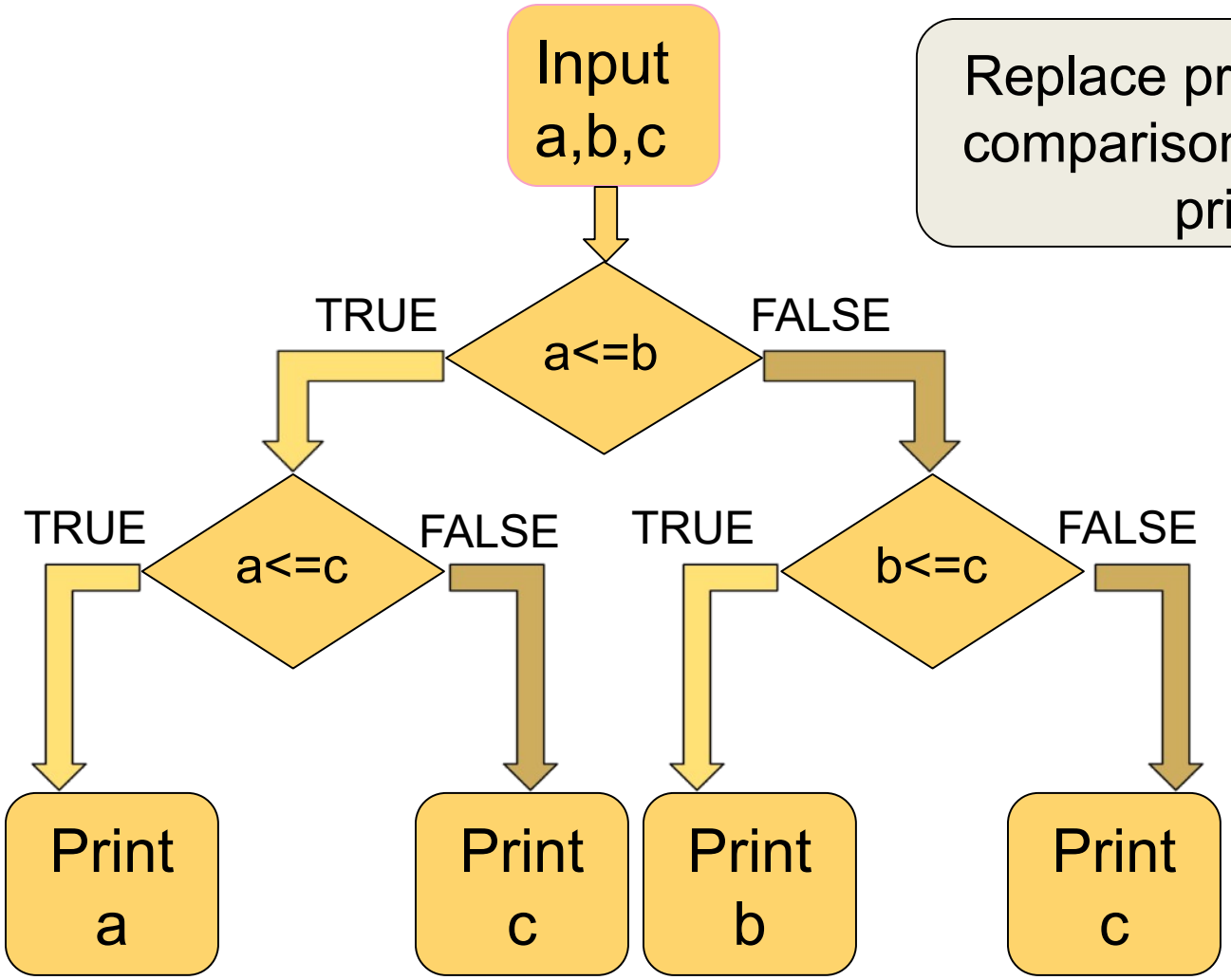
- Each branch translates to an if-else statement
- Hierarchical branches result in nested if-s

More Conditionals

- Sorting a sequence of numbers (i.e., arranging the numbers in ascending or descending order) is a basic primitive.

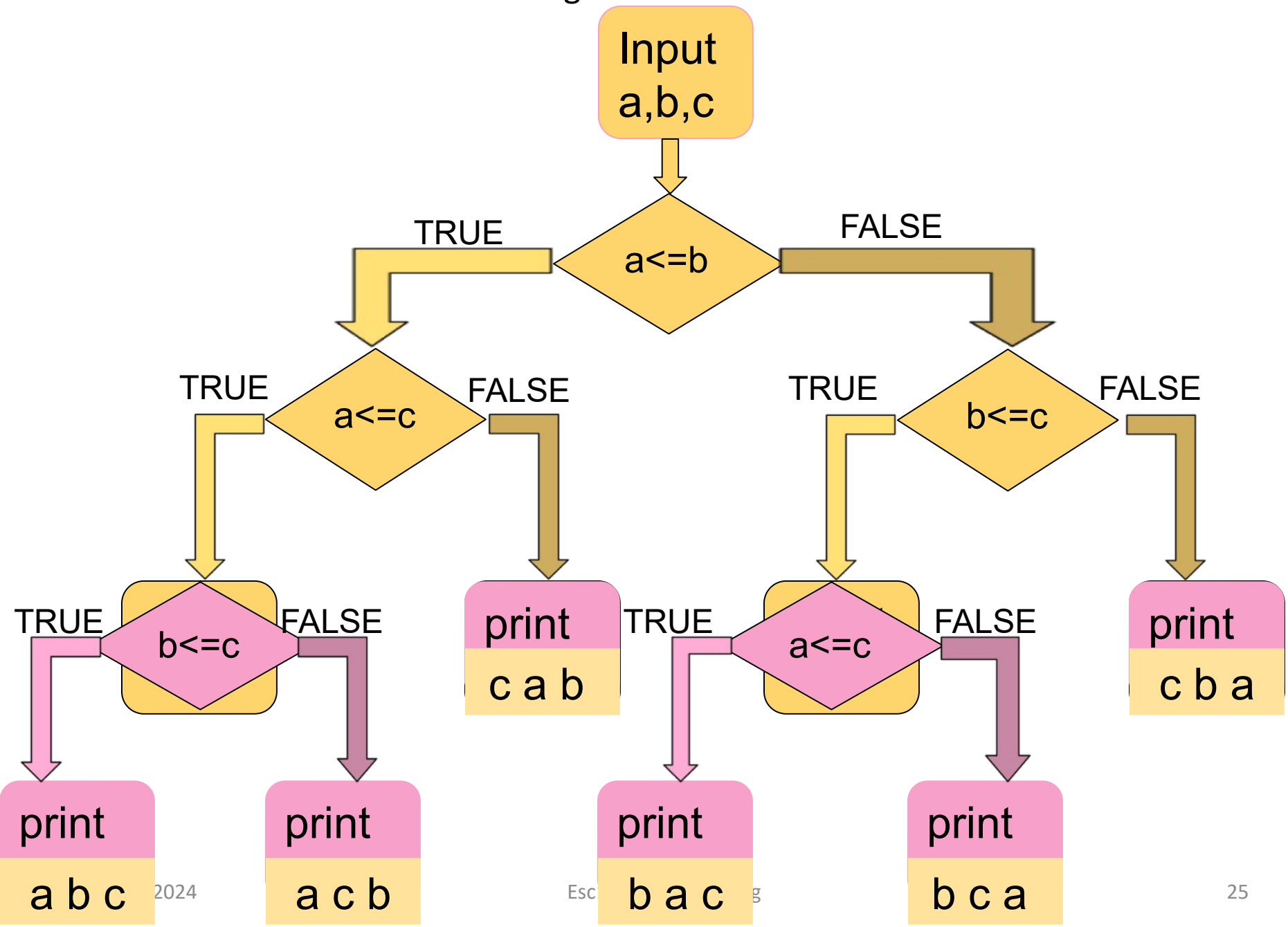
- Problem: read three numbers into a, b and c and print them in ascending order.
  - Start with the flowchart for finding minimum of three numbers and add one more level of conditional check.
  - Then translate the flowchart into C program.

Finding min of 3 numbers



Replace print by more comparisons and then print.

Ascending order of 3 numbers



```
if (a <= b) {
    if (a <= c) {          /* a <= b and a <= c */
          if (b <= c) {    /* a <= b, a <= c, b <= c */
           printf("%d %d %d \n", a, b, c);
          } else {         /* a <= b, a <= c, c < b */
              printf("%d %d %d \n", a, c, b);
          }
    } else {               /* a <= b, c < a*/
          printf("%d %d %d \n", c, a, b) ;
       }
} else {                   /* b < a */
    if (b <= c) {          /* b < a and b <= c */
          if (a <= c) {    /* b < a, b <= c, a <= c */
           printf("%d %d %d\n", b, a, c);
       } else {            /* b < a, b <= c, c < a */
              printf("%d %d %d\n", b, c, a); }
       }
    } else {               /* b < a, c < b */
        printf("%d %d %d\n", c, b, a); }
    }
}
```

Nested if, if-else

- Earlier examples showed us *nested* if-else statements

```
if (a <= b) {
      if (a <= c) { … }  else {…}
} else {
       if (b <= c)  { … } else { … }
}
```

- Because if and if-else are also statements, they can be used anywhere a statement or block can be used.

Summary of if, if-else

- if-else, nested if's, else if.
- Braces {…} can be omitted if a block has only one statement.
- Multiple ways to solve a problem
  - issues of better readability

Next class

- More if-else
- Switch-case
- Examples of conditional statement usage